

Sprint 2 Retrospective – CampusBites

Overview

Sprint 2 focused on extending the system beyond basic authentication to support full interaction between users and vendors. Key features implemented included browsing menu items, cart functionality, order placement, vendor order management, and order status tracking. The team also completed UML diagrams to document the system architecture using the 4+1 model.

What Went Well

- The team successfully implemented the core ordering workflow, including:
 - Browse menu
 - Add to cart
 - Place order
 - Vendor receives and processes orders
 - Order status updates visible to users
- Integration with Firebase services (Authentication, Firestore, Storage) was maintained across multiple features
- The system architecture was clearly defined and documented through UML diagrams:
 - Class Diagram
 - Component Diagram
 - Deployment Diagram
 - Sequence Diagram
 - Use Case Diagram
- Improved use of GitHub Projects:
 - Tasks were tracked using a Kanban board
 - Work was divided into smaller user stories
 - PR workflow improved compared to Sprint 1

- Team collaboration improved:
 - Faster PR reviews
 - Better distribution of tasks
 - Increased support between members
-

What Did Not Go Well

- Testing and code coverage were challenging:
 - Difficulty testing Firebase-related functionality
 - Some components were not adequately tested early
 - Merge conflicts and integration issues occurred:
 - Multiple members worked on overlapping files
 - Some features were not properly integrated initially
 - Bugs and inconsistencies:
 - Missing links between HTML and JavaScript files
 - Incorrect data handling (e.g. IDs not saving correctly)
 - Some features were assumed to work without verification
 - Communication gaps:
 - Occasional duplication of work
 - Delays in identifying and resolving issues
-

What Can Be Improved

- Improve testing strategy:
 - Design code to be testable from the beginning
 - Increase code coverage throughout the sprint, not only at the end
- Improve Git workflow and coordination:
 - Pull latest changes before starting work
 - Avoid working on the same files simultaneously
 - Ensure PRs are reviewed quickly and thoroughly

- Improve validation and debugging practices:
 - Test features locally before committing
 - Ensure full functionality before submitting PRs
 - Improve communication and planning:
 - More frequent check-ins (daily scrum style)
 - Clear ownership of tasks
 - Better alignment before starting development
-

Architectural Decisions

The team adopted a frontend + Firebase architecture, where:

- The frontend (HTML, Tailwind CSS, JavaScript) handles UI and client-side logic
- Firebase services handle backend functionality:
 - Authentication (user login and registration)
 - Firestore (data storage for users, orders, and menu items)
 - Storage (image uploads)

The system was structured around a role-based model:

- Customer
- Vendor
- Admin

A modular design approach was used:

- Separate JavaScript files for each feature (e.g. login, browse, orders)
- Improves maintainability and scalability

The UML diagrams reflect these decisions by showing:

- Data relationships (Class Diagram)
 - System structure (Component Diagram)
 - Interaction flow (Sequence Diagram)
 - Deployment environment (Deployment Diagram)
-

Action Items for Sprint 3

- Improve testing and aim for higher code coverage earlier in development
 - Refine Git workflow to reduce merge conflicts
 - Strengthen communication and coordination within the team
 - Ensure all features are fully tested before integration
 - Adapt system design to accommodate new requirements introduced in Sprint 3
-

Key Takeaway

Sprint 2 marked a significant progression from a basic system to a functional application with real user–vendor interaction. While challenges were encountered in testing, integration, and coordination, the team improved in structure and collaboration. These lessons provide a strong foundation for a more efficient and stable Sprint 3.